COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	AUGUST 22 2026	**DATE SUBMITTED:**	AUGUST 26 2026
NAME:	DUSTIN CARL B. MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 6.1 **Hands-on Activity 6.1 Searching Techniques**

1. PROCEDURE OUTPUT

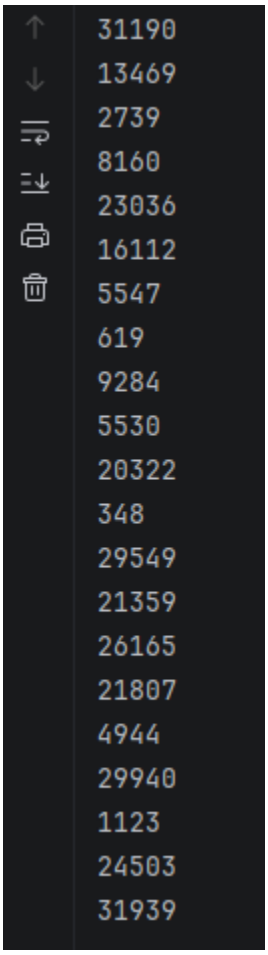
Screenshot	
Observation	The data generated are a bunch of random seed numbers.

Table 6-1. Data Generated and Observations.

Code	HeaderFile:
	`template <typename T>`
	`void linearSearchArr(int arrSize,T dataFind,T list[]) {`
	
	`    //step 1: I = 0`
	`    int i = 0;`
	`    //repeat while I <= n`
	`    while (i < arrSize) {`
	`        if (dataFind == list[i]) {`
	`            std::cout <<"Searching is successful\n";`
	`            return;`
	`        }`
	`        i++;`
	`    }`
	`    //Print unsuccessful`
	`    std::cout <<"Searching is Unsuccessful"<<std::endl;`
	`}`
	
	Main:
	`    //generate`
	`    int dataset[max_size];`
	`    srand( Seed: time(0));`
	`    for (int i = 0; i < max_size; i++) {`
	`        dataset[i] = rand();`
	
	`    }`
	`    for (int i = 0; i < max_size; i++) {`
	`        std::cout << dataset[i] << std::endl;`
	`    }`
	`    std::cout << std::endl;`
	
	`    int userInput;`
	
	`    std::cout<<"What number to search? ";`
	`    std::cin >> userInput;`
	`    std::cout <<"Linear Search result ARR: " << std::endl;`
	`    linearSearchArr(max_size, dataFind: userInput, dataset);`

Output	1123 24503 31939 What number to search?31939 Linear Search result ARR: Searching is successful
Observation	The searching is successful because the number inputted by the user was found in the random seed generated numbers. If the inputted number was not found then the outcome will be "Searching is unsuccessful".

Table 6-2a. Linear Search for Arrays

Code	HeaderFile:
	`template <typename T>`
	`void linearSearchLL(Node<T>* head, T dataFind) {`
	
	`    while (head != nullptr) {`
	`        if (head->data == dataFind) {`
	`            std::cout <<"Searching is successful\n";`
	`            return;`
	`        }`
	`        head = head->next;`
	`    }`
	`    std::cout <<"Searching is unsuccessful"<<std::endl;`
	`}`
	
	MainFile:

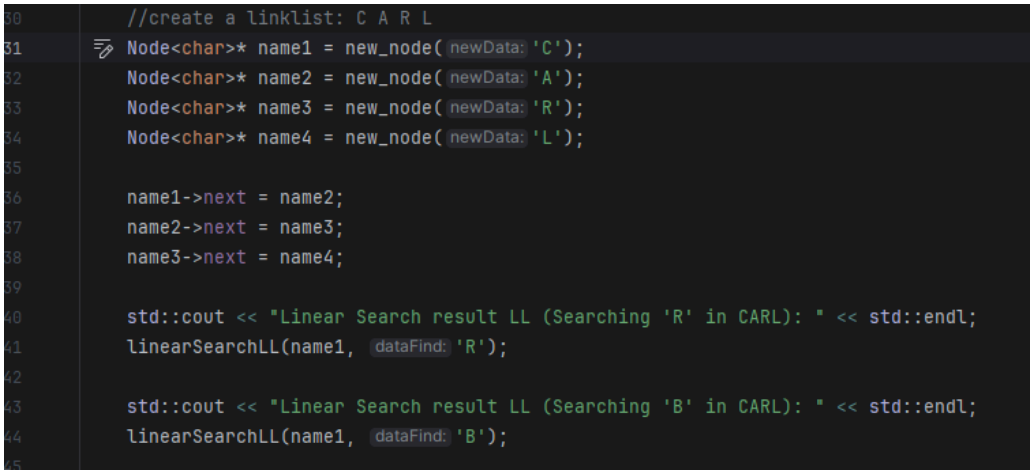
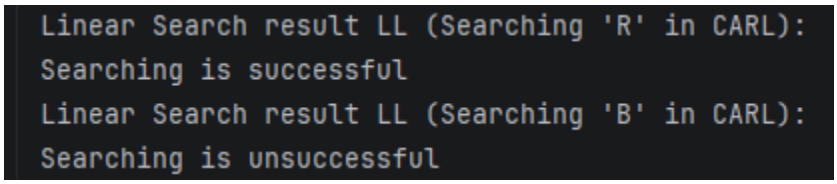
	
Output	
Observation	The first linear searching result was successful because it was searching for 'R' which was present in the name CARL. The last searching result was unsuccessful because 'B' is not present in the name.

Table 6-2b. Linear Search for Linked List.

Code	HeaderFile:

```cpp
//for array binary search
template <typename T>
void binarySearchArr(int arrSize,T dataFind, T list[]) {
    //define low high and middle
    int low = 0, up = arrSize - 1, middle;
    //loop while low iis less than or equal to high
    while (low <= up) {
        //compute for middle
        middle = (int((low + up)) / 2);
        //if data found is middle -> data found -> return
        if (dataFind == list[middle]) {
            std::cout <<"Searching is successful\n";
            return;
        }
        // if data find < data middle ->new high
        else if (dataFind < list[middle]) {
            up = middle - 1;

        }
        //else update low = middle + 1
        else {
            low = middle + 1;
        }

    }
    //search is not found
    std::cout <<"Searching is unsuccessful"<<std::endl;

}
```

MainFile:

```cpp
//Testing binary serach array
const int sortedArrSize = 5;
int sortedDataset[sortedArrSize] = {11, 12, 13, 14, 15};

std::cout << "\nBinary Search result ARR: " << std::endl;
std::cout << "What number to search in the sorted array (11, 12, 13, 14, 15)? ";
std::cin >> userInput;
binarySearchArr(sortedArrSize, dataFind: userInput, sortedDataset);
```

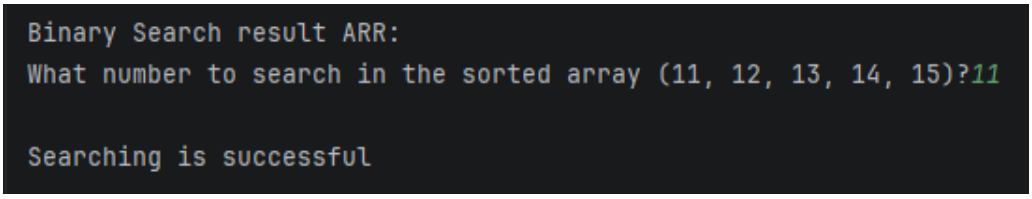
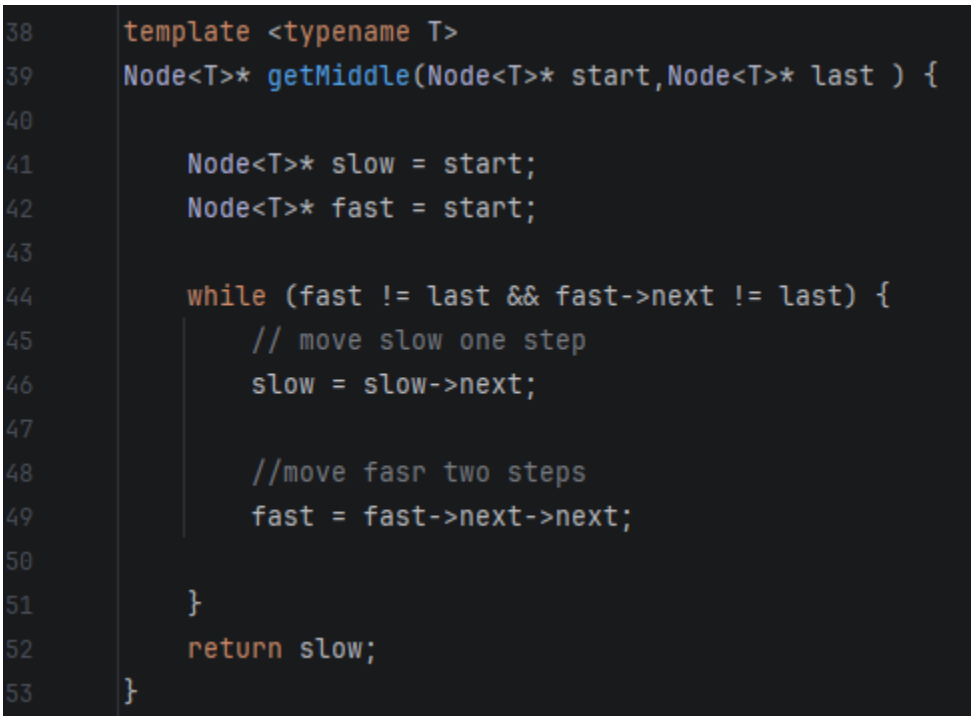
Output	
Observation	In this output the searched number was 11 which was present in the dataset thus the search was successful. But if the inputted number was not in the dataset, the search will be unsuccessful. Also it is worth keeping in mind that binary search only works on sorted data.

Table 6-3a. Binary Search for Arrays

Code	HeaderFile: 

```cpp
template <typename T>
void binarySearchLL(Node<T>* start, T dataFind) {
    Node<T>* last = nullptr;

    while (start != last) {
        Node<T>* middle = getMiddle(start, last);
        if (middle == nullptr) {
            return;
        }
        if (middle->data == dataFind) {
            std::cout <<"Searching is successful\n";
            return;
        }
        else if (middle->data > dataFind) {
            last = middle;

        }
        else {
            start = middle->next;
        }
    }
    std::cout <<"Searching is unsuccessful"<<std::endl;


}
```

MainFile:

```cpp
//Testing binary search linked list
Node<int>* sortedList = new_node( newData: 1);
sortedList->next = new_node( newData: 2);
sortedList->next->next = new_node( newData: 3);
sortedList->next->next->next = new_node( newData: 4);

std::cout << "\nBinary Search result LL (Searching 2 in 1-2-3-4): " << std::endl;
binarySearchLL(sortedList,  dataFind: 2);
```

Output

```
Binary Search result LL (Searching 2 in 1-2-3-4):
Searching is successful
```

Observation	The binary search was successful because the number 2 was found in the linked list data. The search will be unsuccessful if the number is not in the data or that the data is not sorted.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1:
Code:

```
    std::cout << "Problem #1" << std::endl;

    //array
    int p1List[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int p1Size = 10;

    std::cout << "Array Approach (Searching for 18): ";
    linearSearchArr(p1Size, dataFind: 18, p1List);

    //LL
    Node<int>* Head = new_node( newData: 15);
    Head->next = new_node( newData: 18);
    Head->next->next = new_node( newData: 2);
    Head->next->next->next = new_node( newData: 19);
    Head->next->next->next->next = new_node( newData: 18);
    Head->next->next->next->next->next = new_node( newData: 0);
    Head->next->next->next->next->next->next = new_node( newData: 8);
    Head->next->next->next->next->next->next->next = new_node( newData: 14);
    Head->next->next->next->next->next->next->next->next = new_node( newData: 19);
    Head->next->next->next->next->next->next->next->next->next = new_node( newData: 14);

    std::cout << "Linked List Approach (Searching for 18): ";
    linearSearchLL(Head, dataFind: 18);
    std::cout << std::endl;
```

Output:

```
Problem #1
Array Approach (Searching for 18): Searching is successful
Linked List Approach (Searching for 18): Searching is successful
```

- It took 2 comparisons to find '18' since the sequential search checks the first element, then checks the second element which is exactly '18'. Thus finding the match on the second comparison.

Problem 2:
HeaderFile:

```
111  template <typename T>
112  int countInstancesArr(int arrSize, T dataFind, T list[]) {
113      int count = 0;
114      int i = 0;
115      while (i < arrSize) {
116          if (dataFind == list[i]) {
117              count++;
118          }
119          i++;
120      }
121      return count;
122  }
```

Main:

```
31      std::cout << "Problem #2" << std::endl;
32      int problem2List[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
33      int p2Size = 10;
34
35      int instances = countInstancesArr(p2Size, dataFind: 18, problem2List);
36      std::cout << "Number of times 18 appears: " << instances << std::endl;
37      std::cout << std::endl;
```

Output:

```
Problem #2
Number of times 18 appears: 2
```

- The number 18 appears 2 times because there are 2 '18' in the dataset.

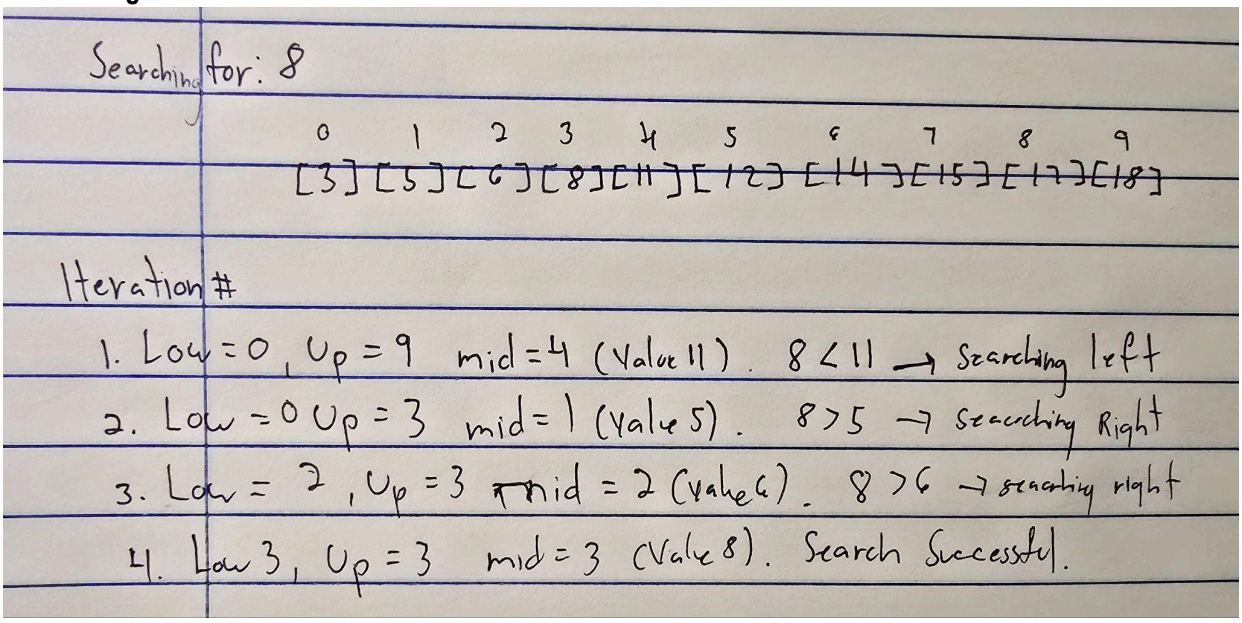
Problem 3:
Code:

```
39        std::cout << "Problem #3" << std::endl;
40        int sortedList[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
41        int p3Size = 10;
42
43        std::cout << "Searching for 8 using standard Binary Search: ";
44        binarySearchArr(p3Size,  dataFind: 8, sortedList);
45        std::cout << std::endl;
```

Output:

```
Problem #3
Searching for 8 using standard Binary Search: Searching is successful
```

Written Diagram:



Problem 4:
HeaderFile:

```
124     template <typename T>
125     void recursiveBinarySearchArr(int low, int up, T dataFind, T list[]) {
126         if (low <= up) {
127             int middle = (low + up) / 2;
128             if (dataFind == list[middle]) {
129                 std::cout <<"Searching is successful\n";
130                 return;
131             }
132             else if (dataFind < list[middle]) {
133                 recursiveBinarySearchArr(low, up: middle - 1, dataFind, list); //left
134             }
135             else {
136                 recursiveBinarySearchArr( low: middle + 1, up, dataFind, list); //right
137             }
138         }
139         else {
140             std::cout <<"Searching is unsuccessful\n";
141         }
142     }
```

Main:

```
47          std::cout << "Problem #4" << std::endl;
48          std::cout << "Searching for 8 using recursive Binary Search: ";
49          recursiveBinarySearchArr( low: 0, up: 9, dataFind: 8, sortedList);
50          std::cout << std::endl;
```

Output:

```
Problem #4
Searching for 8 using recursive Binary Search: Searching is successful
```

Conclusion:

I therefore conclude that this activity helped me and guided me to create a code for searching data using different techniques. And solve different problems utilizing appropriate searching techniques in C++. Furthermore I also did an observation for my output in the procedure part and for the analysis of my supplementary activity. I did what was asked in the problems and created a program to solve the problems and modified my existing codes just like in the header file so that I can implement a modified searching algorithm techniques like a recursive binary search array wherein it repeatedly halves a sorted array by having boundaries until it finds the target value or until the elements runs out. Lastly, I will keep learning to enhance my programming skills.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ai to deepen my understanding and to guide me.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.